

Foundations of Distributed Systems

Implementing Group Comm, Remote Procedure Call

Chapter 3.3 / 2025-10-03

Prof. Dr. Christian Tschudin

Dept of Mathematics and Computer Science, University of Basel

Overview

Context: “Group Communication” as an abstraction layer

- Group Communication:
 - Reliable Broadcast
 - **Replication**
 - More on Broadcast: atomic BC, generic BC
- Remote Procedure Call Semantics
- A RPC protocol from 1987

Distributed Algorithms: Replication

Overall assessment of fault tolerance (in distr. systems)

- Transactions, checkpointing:
 - enables recovery from node crashes
 - uses redundancy in time (redo events after recovery)
- Replication:
 - *masks* node crashes
 - use hardware redundancy

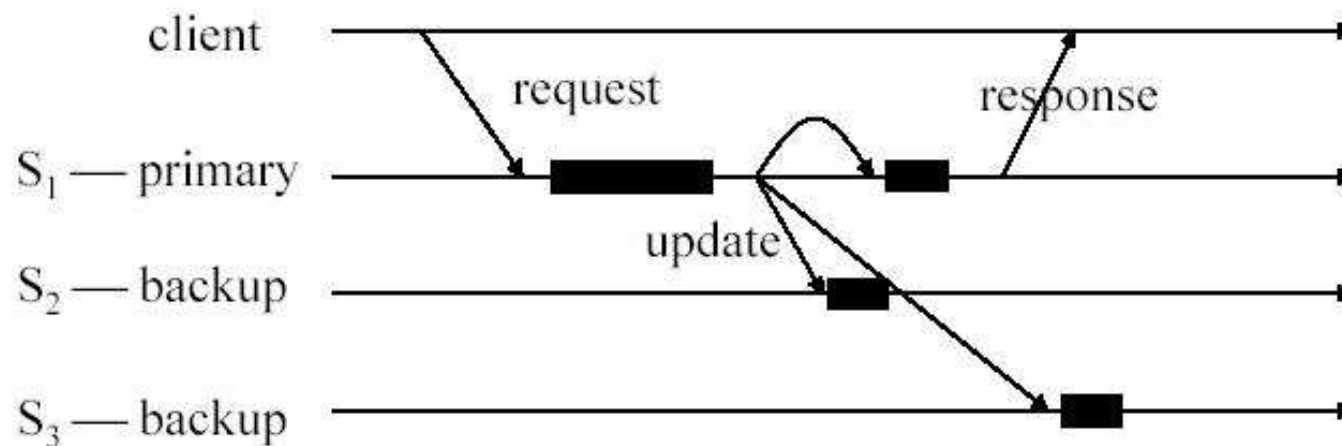
Passive vs Active Replication

- passive:
 - one *primary protocol engine*, many secondary sites for logs
- active:
 - symmetric, *parallel protocol engines*

Both approaches have their advantages, disadvantages

Replication: passive

Passive replication, also called *primary backup replication*

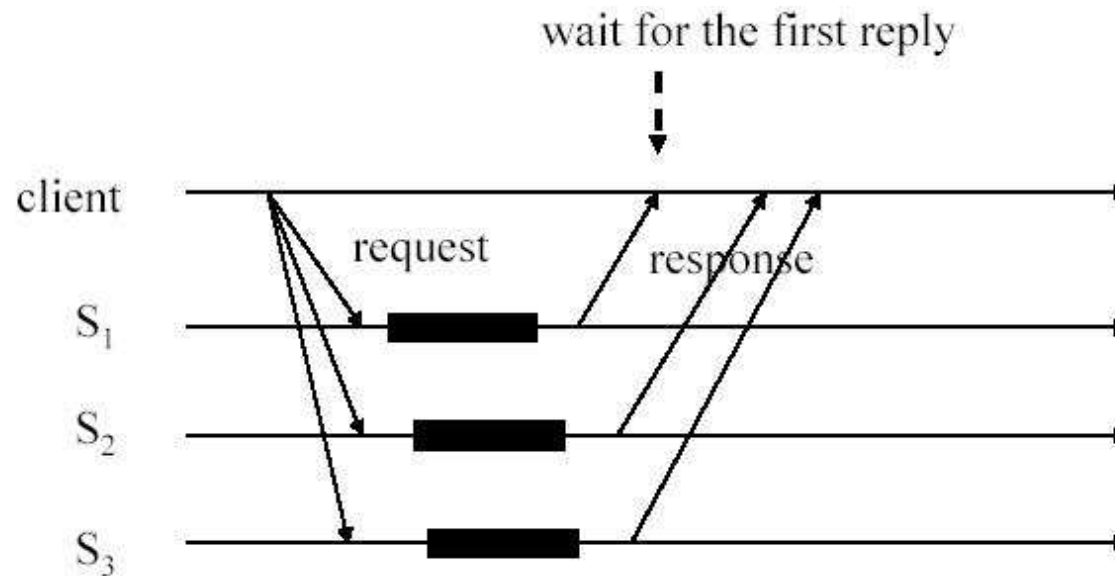


(Fig: Schiper, 2006)

- crash is “decided” based on time-outs
- (we need to address faulty crash decisions)

Replication: active

Active replication: many servers run the (full) protocol



(Fig: Schiper, 2006)

- a client waits for first response, discards the others
- server crash is transparent to clients

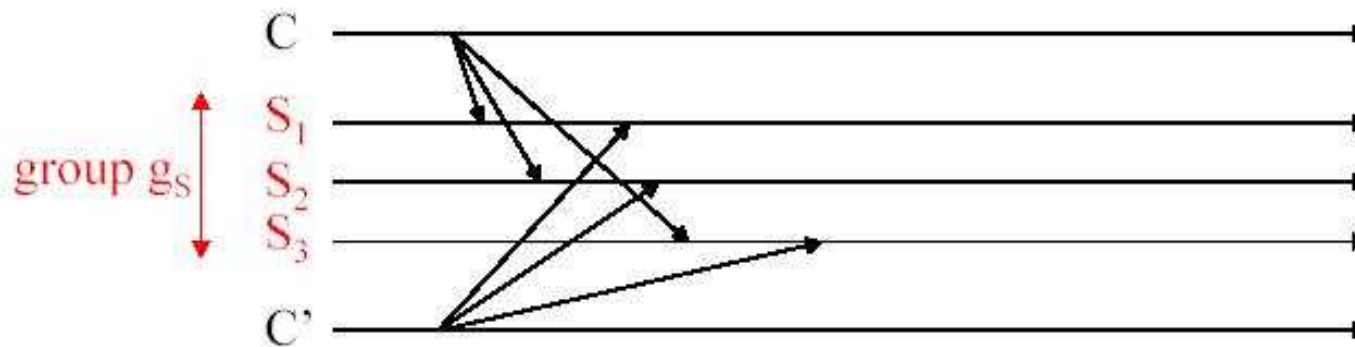
Overview

- Group Communication:
 - Reliable Broadcast
 - Replication
 - **More on Broadcast: atomic BC, generic BC**
- Remote Procedure Call Semantics
- A RPC protocol from 1987

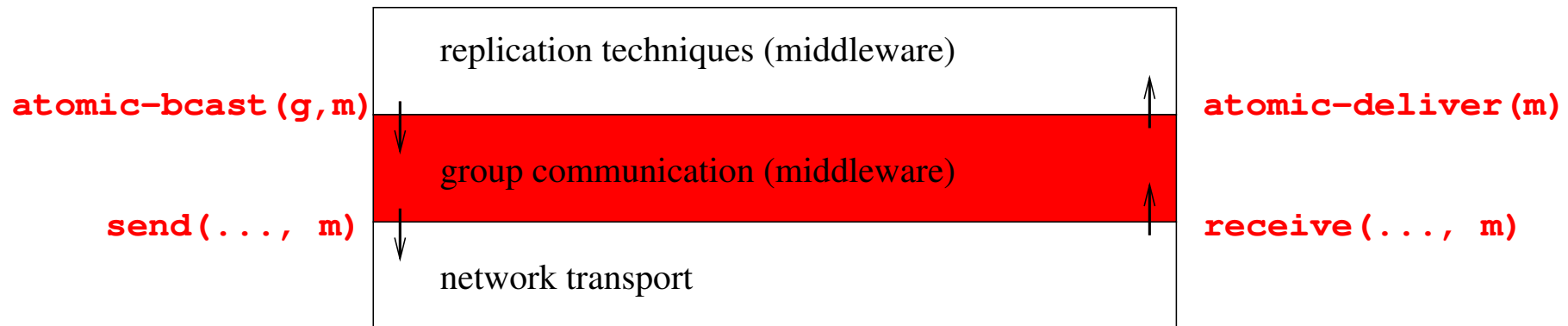
Best Broadcast Primitives (for Replication)?

Active replication with many clients: server_s must receive requests in the same order (in order to compute the same response)

- Useful group communication primitive: **atomic broadcast** (also called total order broadcast)
- The set of replicated servers form a group



Group Communication: (Atomic) Broadcast



- Group communication layer on top of network transport (UDP)
- Group communication provides **atomic or generic broadcast**, possibly using network layer broadcast, or using unicast
- atomic broadcast suitable for active replication
(passive replication: “generic” broadcast” sufficient, see later)

Atomic Broadcast: Specification

Players: Group g , processes p, q , messages m, m'

- If process p executes **atomic-bcast(g, m)** and does not crash, then all processes in g eventually do **atomic-deliver(m)**
- If some process in g does **atomic-deliver(m)** and does not crash, then all processes in g that do not crash eventually do **atomic-deliver(m)**
- If two processes p, q in g both do **atomic-deliver(m)** and **atomic-deliver(m')**, then they do it in the same order.

Side Note 1 on *Generic* Broadcast

Generic broadcast: same spec as atomic broadcast, except that messages may be delivered in different order.

- Generic broadcast based on a conflict relation on the messages:
 - “conflicting messages” are ordered
 - non conflicting messages are not ordered – not detailed here.

In order to implement *passive* replication:

- we *could* also use atomic broadcast
- **but** generic broadcast more efficient (and sufficient)

Side Note 2 on “How to handle Crash?”

- **“Fail-stop”**
 - complete node state is lost after crash (no persistent storage)
(not so useful in practice: eventually, all nodes are dead)
- Fail-Stop and dynamic group:
 - static group: group gets extinct after n crashes
 - allow to add replacement nodes (dynamic group)
 - even with dynamic group: system not tolerant to catastrophic failure (all nodes crash at the same time)

In practice: Dynamic groups with crash-stop model

Back to Implementing Atomic Broadcast

Atomic as well as generic broadcast contain a consensus problem: Members have to agree on a **sequence of message sets**, we number these sets.

- Each process p has: counter k , set s of undelivered messages
- Upon `atomic-bcast( $m$ )` do `broadcast( $m$ )`
- Upon `receive( $m$ )` do
 - add m to s
 - **if** no consensus algo running **then**
 - $M(k) = \text{consensus}(s)$
 - foreach m in $M(k)$: `atomic-deliver( $m$ )` in common order
 - $k = k+1$

(Difference between Causal vs Total Order)

How should messages be delivered, in group communications?

- Total order:

All messages that are multicast to a group are delivered to all members of the group in the same order.

(Possible implementation: central server, or consensus)

- Causal order (see logical clock discussion):

Causally related messages from multiple sources are delivered in causal order.

Other messages can be delivered differently, for some members.

Two weaker cases: FIFO (delivery order like sent), or unordered.

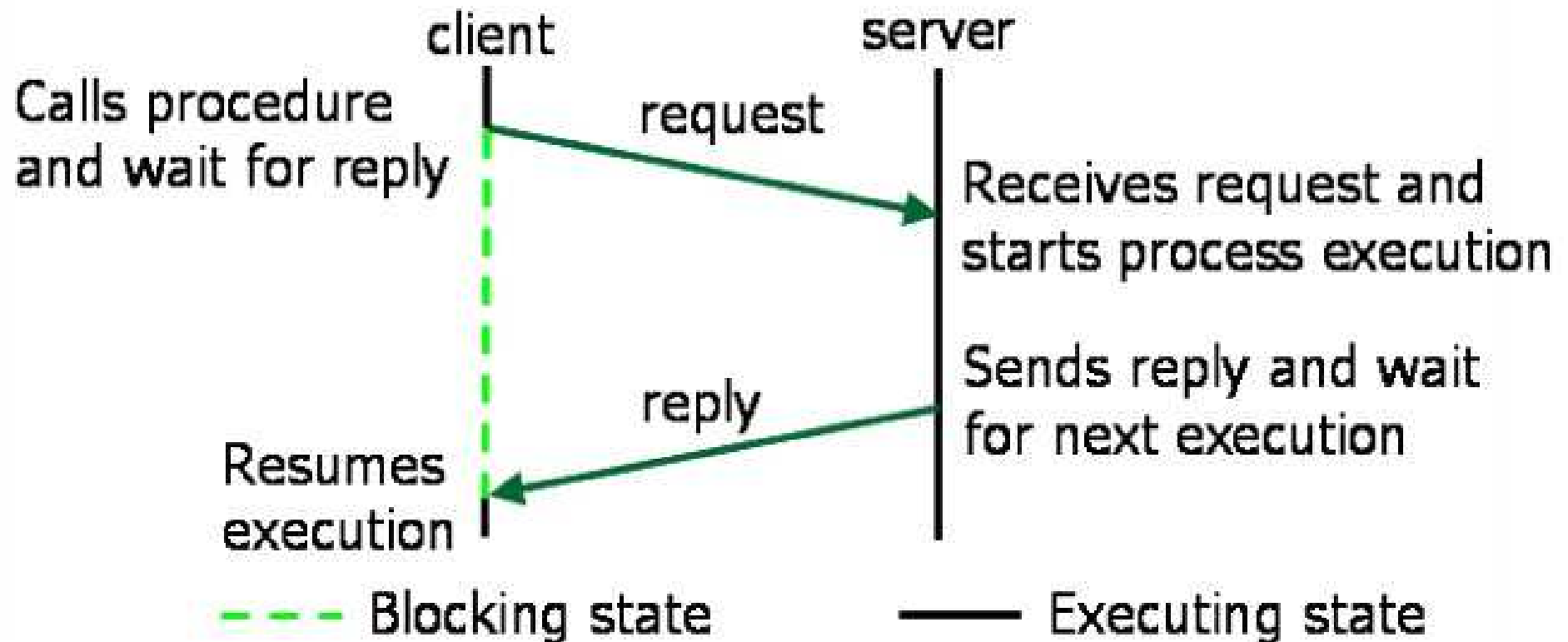
Group Communication (concluding summary)

- Theory is well advanced on static group and crash-stop model, same for implementations
- Still research on-going for other combinations:
 - mostly dynamic group/crash-stop
- Performance an issue, comparing various atomic broadcast algorithms, other set of primitives.
- Link to HPC libraries (e.g. message passing interface MPI)

Overview

- Group Communication:
 - Reliable Broadcast
 - Replication
 - More on Broadcast: atomic BC, generic BC
- **Remote Procedure Call Semantics**
- A RPC protocol from 1987

Remote Procedure Call – Intro

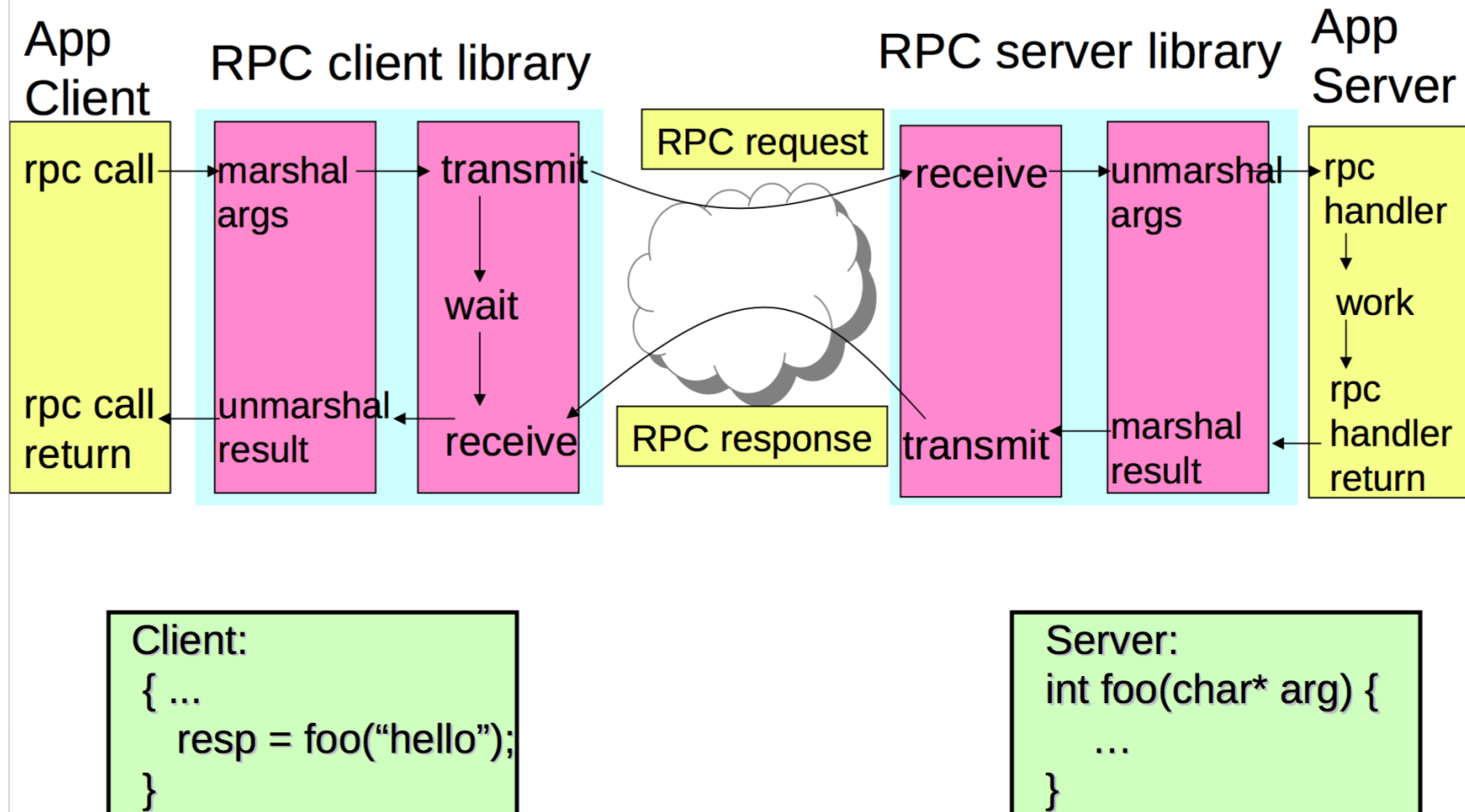


RPC History

- Networking had to become “normal” for RPC to be invented:
 - 1981 Nelson’s PhD thesis
- Main goal:
 - ease of (distributed) programming
 - client server model, but hiding the socket interface
- “Ease” includes automating auxiliary tasks:
 - marshalling
 - (code) stub generation

(subsequent slides/figures copied from Peter Du, Columbia, USA)

RPC Intro (a more fine-grained look)



RPC Marshaling

Calling and called procedures run on different machines, with different address spaces.

- pointers are meaningless (linked lists...?)
- different operating systems, memory layout e.g.: endian problem
- Marshaling:
 - Must convert* from sender's local data representation
 - ... to network representation
 - ... and back to receiver's local representation

Do you remember `htonl()` ?

RPC Stubs and IDLs

RPC stubs are automatically generated procedures that appear to implement the desired functions

They just do marshalling/unmarshalling and then request transmission, watch over timeouts.

- IDL – Interface Definition Language
- “RPC frameworks” (and their IDLs):
 - SunRPC and XDR (external Data Repr), heavily used in NFS
 - Corba, XML/RPC, SOAP and Ajax, **Java RMI**
 - Google’s **ProtoBuf and gRPC**, heavily used across Google
 - Facebook’s Thrift

RPC Semantics (I)

What happens in case of communication failures?

Programmer code still has to deal with exceptions!

- Timeouts:

Normally, calling `foo()` to add $5 + 5$ can't fail and doesn't take 10 seconds to return

- Machine failures?

– Did server fail *before or after* processing a request?

Impossible to tell, if it's still down...

It's impossible to hide all of the complexity inside an RPC layer. But marshaling/unmarshaling support is great!

RPC Semantics (II)

Different “RPC semantics”, depending on

- Communication failures
 - delayed and lost messages
 - connection resets
- Machine failures
 - server or client failures
 - did server fail **before or after** processing the request?

Note: from just waiting for a message it's impossible to tell communication failures from machine failures.

RPC Semantics (III)

Semantic of local procedure call is **exactly-once**. RPC case:

- Exactly-once impossible to achieve with RPC:
client will never know outcome if server crashes before replying

Semantic of local procedure call is **exactly-once**. RPC case:

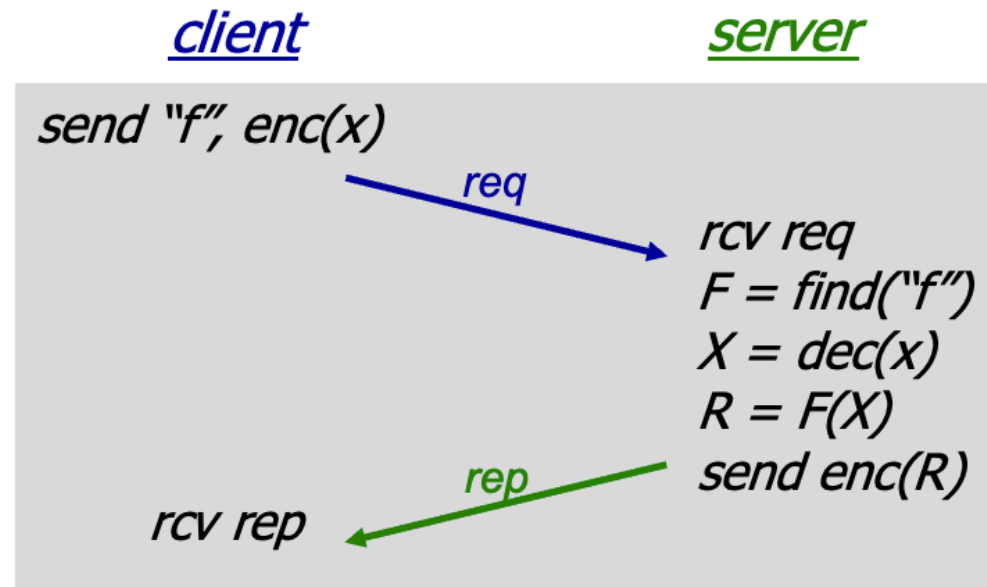
- Exactly-once impossible to achieve with RPC:
client will never know outcome if server crashes before replying
- Instead: **at-least-once**
 - on return from RPC, sure that server saw the request
 - works only if $f(x) = f(f(x))$ (idempotent function)
 - implement: retransmit request on time-out (= *keep trying*)
- Instead: **at-most-once**
 - on return, RPC called exactly once, *or not at all*
 - implement: use seq #, server filters duplicate requests
 - still a problem if server crashes during RPC

RPC Semantics (IV)

implementing at-least-once RPC

Implementing
at-least-once RPC
for a remote function f

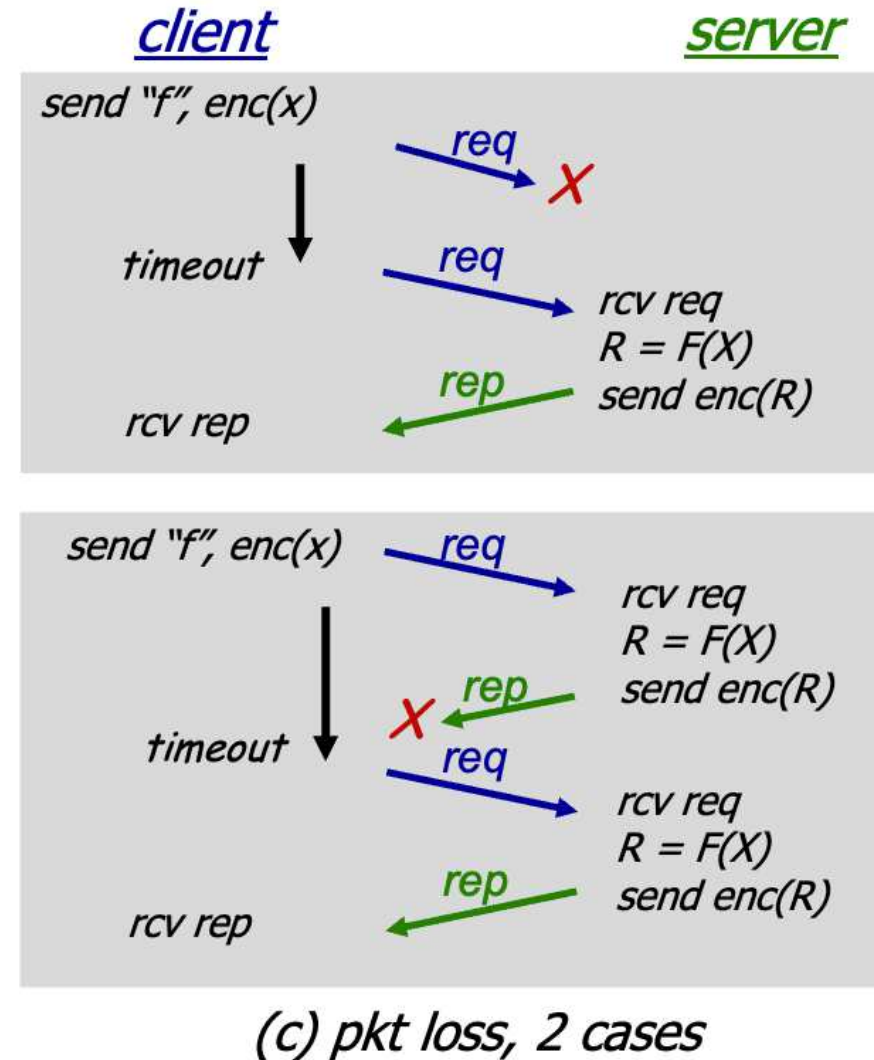
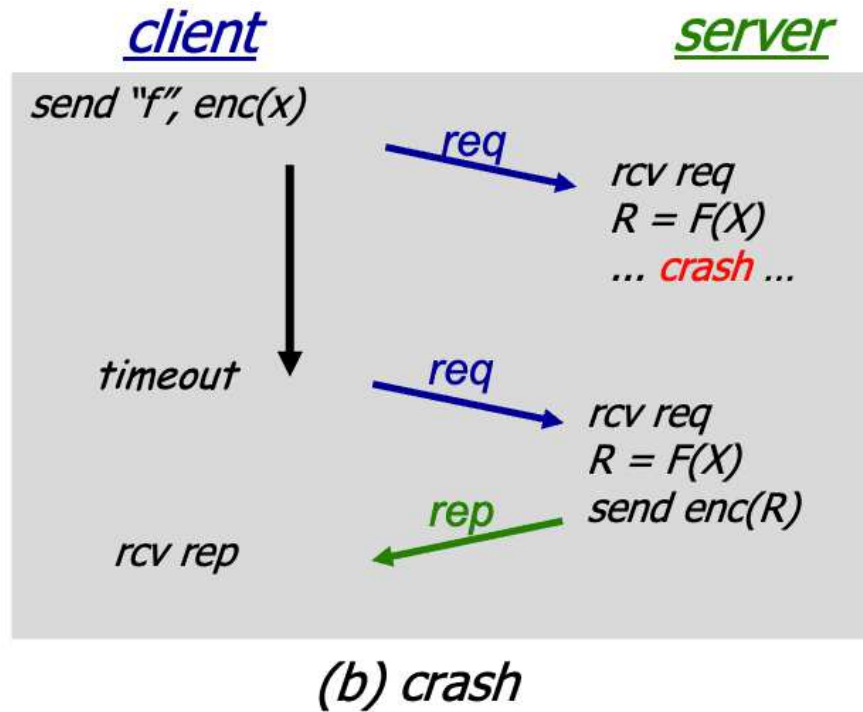
- ❖ A simple ACK scheme is fine:
 - the client resends (after some timeout) the request until the reply is received
 - using UDP is OK



(a) no crash, no pkt loss

RPC Semantics (V)

at-least-once scenarios

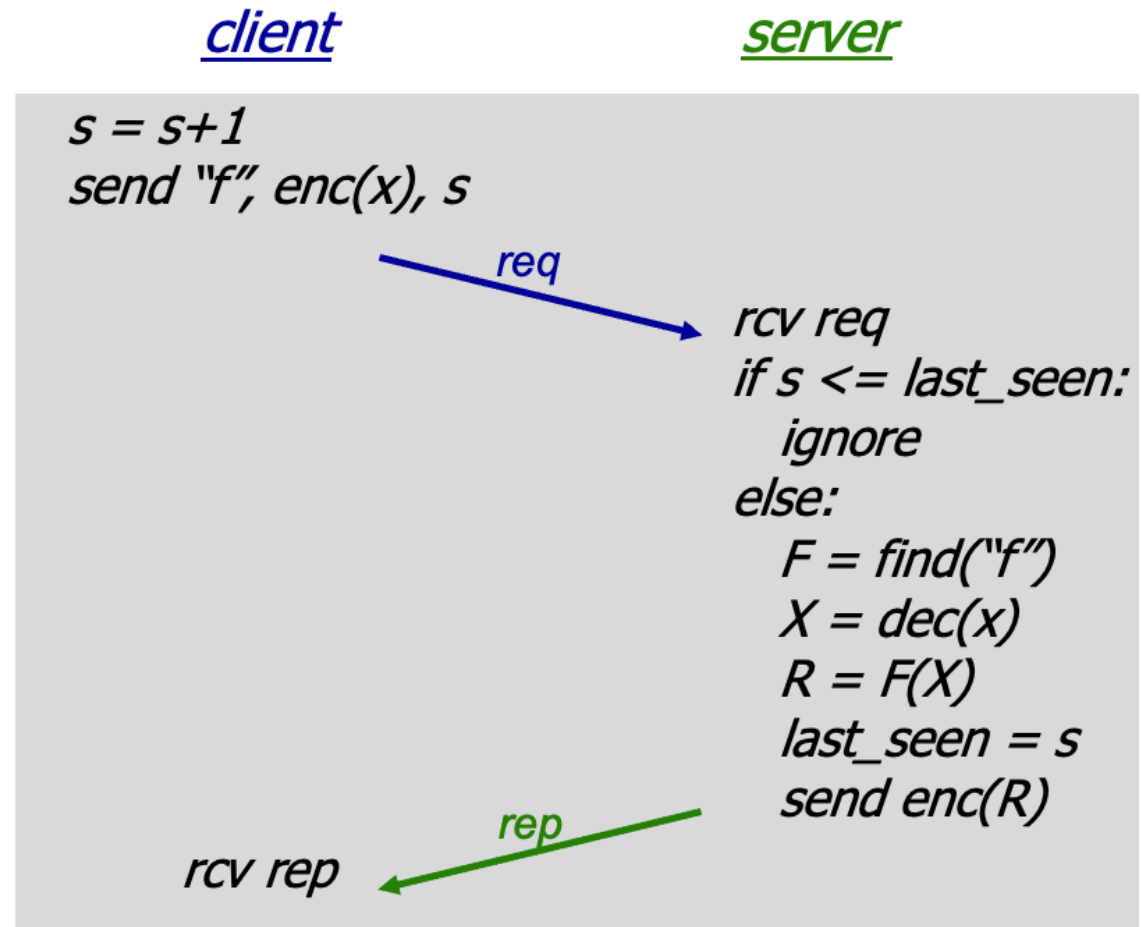


RPC Semantics (VI)

implementing at-most-once RPC

Implementing
at-most-once RPC
for remote function f

- ❖ solution requires server-side state:
 - **client uses unique RPC request ID**
 - server stores all request IDs it has ever served
 - can use per-client sequence number



(a) no crash, no pkt loss

RPC Semantics (VII)

at-most-once: we need atomicity

- ❖ Above solution still has a problem at the server side if crash happens after executing F , but before storing the new value for $last_seen$
- ❖ → WAL (write-ahead-log), a technique for file system consistency

```
rcv req
if  $s \leq last\_seen$ :
    ignore
else:
     $F = find("f")$  ← crash here (1)
     $X = dec(x)$ 
     $R = F(X)$  ← crash here (1)
     $last\_seen = s$  ← crash here (2)
    send enc( $R$ ) ← crash here (3)
```

*Server-side problem: atomicity
(doing action F **and** recording
that it was done)*

Are there situations where we **get** “exactly-once RPC semantics”?

- Yes, if some conditions are met
- **If** we get a positive reply from an at-most-once RPC, then we know for sure that:
 - the call was successfully executed, and
 - the call was executed not more than once, hence exactly once. This is great!
- A second consideration:
 - nested RPCs? yes, that’s fine too.However: “external” side effects are problematic . . .

Why is “exactly-once RPC semantics” impossible, in general?

- Consider a message that triggers an external physical action.
Say: a robot fires a nerf dart at the professor. The robot can crash immediately before or after firing.
- Without reply: Don’t know which one happened → ask repeatedly
- Even with reply: Zooming in to micro-second resolution . . .
does not help. Crashed before pulling the trigger? Or afterwards?
 - writing this state to harddisk every millisecond insufficient
 - no atomicity between recorded state and real-world side-effect
 - would need a sensor (dart still present, or not)

RPC Semantics and Transport

From “ONC, Appendix B”, Section “Transport and Semantics”

... If you tell the RPC service that it is running on top of a reliable transport such as TCP, most of the work is already done for the service. On the other hand, if RPC is running on top of an unreliable transport such as UDP, the service must devise its own retransmission and time-out policy. RPC does not provide this service.

... If an application retransmits RPC messages after short timeouts receiving no reply, it can infer only that the procedure was executed zero or more times. If the application does receive a reply, it can infer that the procedure was executed at least once.

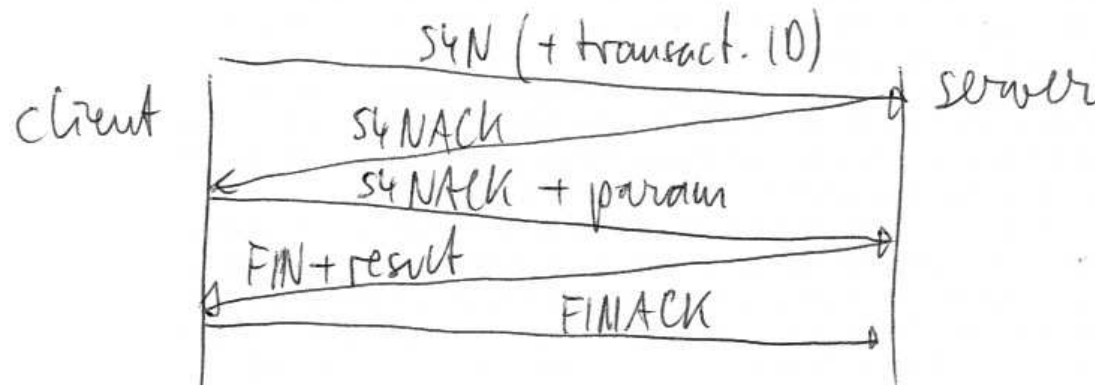
... On the other hand, if using a reliable transport such as TCP, the application can infer from a reply message that the procedure was executed exactly once.

If the application receives no reply message, it cannot assume the remote procedure was not executed.

RPC, a transactional view

How many TCP packets would be required for a single request-response transaction?

A minimum of 5 packets: 3 for the initial handshake, plus 2 for the FIN and FIN ACK (assuming that we can piggy back data and a FIN on the third packet of the 3-way handshake).



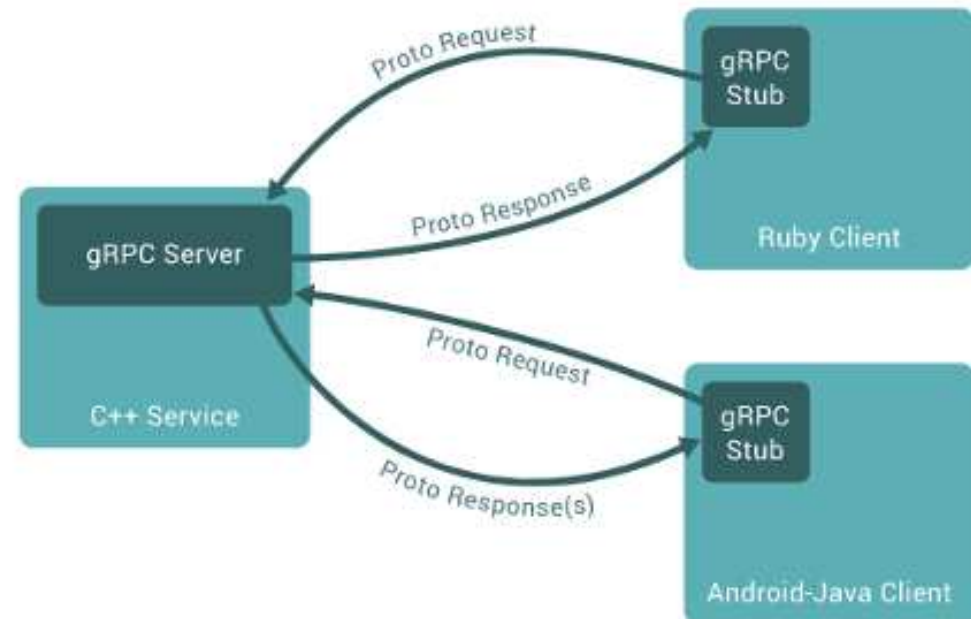
Google's ProtoBuf and gRPC

see <https://grpc.io/docs/what-is-grpc/introduction/>

- Protocol Buffers: a data serialization/encoding format
 - similar to ASN.1, XDR, Thrift, Ion, Bond etc
 - wide range of supported prog languages (C, Java, JS, Python..)
- gRPC: Google's RPC library
 - IDL (to describe the function interfaces)
 - runs on top of TCP (unfortunately, I think)

gRPC continued

- ❖ Cross-platform support a major requirement (for offering cloud-based remote functions)
- ❖ Use the “protoc” compiler to create the RPC stubs and data marshalling functionality



gRPC – a look at the IDL

A *.proto file defines the parameter types as well as the interface, and N:M relation of requests and/or replies (streaming)

- ❖ Example to the right:
 - one remote function
 - two types, one for the request, the other for the response.
- ❖ this is an “unary RPC”:
1 request -> 1 reply

```
service HelloService {  
    rpc SayHello (HelloRequest) returns (HelloResponse);  
}  
  
message HelloRequest {  
    string greeting = 1;  
}  
  
message HelloResponse {  
    string reply = 1;  
}
```

gRPC and streaming

The three streaming cases:

- ❖ one request, but stream of results (1:N)

```
rpc LotsOfReplies(HelloRequest) returns (stream HelloResponse);
```

- ❖ many requests, but only one result (N:1)

```
rpc LotsOfGreetings(stream HelloRequest) returns (HelloResponse);
```

- ❖ stream of requests and stream of responses (N:M)

```
rpc BidiHello(stream HelloRequest) returns (stream HelloResponse);
```

RPC outlook: Web and RESTful

Alternative approach: RESTful (Representational State Transfer)
(Roy Fielding, 2000, <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>)

- Resource access via remote server which only shows “representations”
- client does *not* need to know and track all remote methods (no need to implement local stubs) - just needs their name
- Start with a URL base path, receive HTML documents with additional methods or resources (discovery)
- stateless = **server** treats each request in isolation, does not need to keep state for all its ongoing client “sessions”, “authorization state”
- it's the **client** which is responsible for session state: has to (re-)submit all necessary state with each new request, including authorization tokens